

# PRÁCTICA 6

Elena Abreu Fernández y Tsan-Yu Wu



**UNIVERSIDAD  
DE GRANADA**

# 1. Análisis para room\_type

Hemos comenzado esta tarea con el objetivo de predecir el tipo de habitación del alojamiento a partir del campo “description”. Inicialmente hemos hecho una división del índice de 70% entrenamiento y 30% test. Hemos obtenido la siguiente matriz de confusión para el clasificador KNN:

```
ConfusionMatrix{linearizedMatrix={Private room={Private room=77}, Shared room={Private room=2}, Entire home/apt={Private room=188}}, avgClassificationTime=0.5767790262172284, numberOfEvaluatedDocs=267}
```

A partir de esto podemos estimar que aproximadamente la distribución de las clases es :

- Private room = 77 docs
- Entire home/apt = 188 docs
- Shared room = 2 docs

Por tanto, partimos de un conjunto de datos desbalanceado, lo cuál nos ayuda a entender los resultados que obtenemos.

Primero usamos el clasificador SimpleNaiveBayes y obtenemos :

```
=== TAREA: roomtype :: SimpleNaiveBayesClassifier ===
Confusion Matrix:
ConfusionMatrix{linearizedMatrix={Private room={2=25, privat=18, apt=16, share=17, room=1}, Shared room={share=1, room=1}, Entire home/apt={2=96, apt=53, privat=1, share=38}}, avgClassificationTime=6.797752808988764, numberOfEvaluatedDocs=267}
Clase: Private room
  Precision(Private room) = 0.0
  Recall(Private room)    = 0.0
  F1(Private room)        = 0.0
Clase: Shared room
  Precision(Shared room) = 0.0
  Recall(Shared room)    = 0.0
  F1(Shared room)        = 0.0
Clase: Entire home/apt
  Precision(Entire home/apt) = 0.0
  Recall(Entire home/apt)    = 0.0
  F1(Entire home/apt)        = 0.0
Clase: 2
  Precision(2) = 0.0
  Recall(2)    = 0.0
  F1(2)        = 0.0
Clase: privat
  Precision(privat) = 0.0
  Recall(privat)    = 0.0
  F1(privat)        = 0.0
Clase: apt
  Precision(apt) = 0.0
  Recall(apt)    = 0.0
  F1(apt)        = 0.0
Clase: share
  Precision(share) = 0.0
  Recall(share)    = 0.0
  F1(share)        = 0.0
Clase: room
  Precision(room) = 0.0
  Recall(room)    = 0.0
  F1(room)        = 0.0
Docs evaluados = 267
```

¡El clasificador no acierta ningún documento! Si además nos fijamos en la matriz de confusión vemos que las etiquetas aparecen tokenizadas y no completas, lo que nos indica que tenemos que corregir varias cosas.

Observamos resultados muy parecidos para BM25.

Por último, para KNN con K=15 obtenemos:

```
=== TAREA: roomtype :: KNearestNeighborClassifier ===
Confusion Matrix:
ConfusionMatrix{linearizedMatrix={Private room={Private room=77}, Shared room={Private room=2}, Entire home/apt={Private room=188}}, avgClassificationTime=0.5767790262172284, numberOfEvaluatedDocs=267}
Clase: Private room
  Precision(Private room) = 0.2883895131086142
  Recall(Private room)    = 1.0
  F1(Private room)        = 0.4476744186046511
Clase: Private room
  Precision(Private room) = 0.0
  Recall(Private room)    = 0.0
  F1(Private room)        = 0.0
Clase: Shared room
  Precision(Shared room) = 0.0
  Recall(Shared room)    = 0.0
  F1(Shared room)        = 0.0
Clase: Entire home/apt
  Precision(Entire home/apt) = 0.0
  Recall(Entire home/apt)    = 0.0
  F1(Entire home/apt)        = 0.0
Clase: 2
  Precision(2) = 0.0
  Recall(2)    = 0.0
  F1(2)        = 0.0
Docs evaluados = 267
```

Es decir, sólo predice Private room : el clasificador no está equilibrado.

Para resolver estas problemáticas tomamos las siguientes medidas:

- Normalizamos las etiquetas de room\_type pasándolas todas a minúscula y convirtiéndolas de manera que Private room = private\_room
- Cambiamos EnglishAnalyzer por StandardAnalyzer, para ver si un stemming menos agresivo nos proporciona mejores predicciones.
- Probamos un k más pequeño, concretamente k = 5.
- Aumentamos la limpieza de los datos de description

Como hemos visto, los datos están desequilibrados (tan solo dos shared\_room), pero con estos ajustes ya obtenemos resultados más interesantes:

El clasificador SimpleNaiveBayes imprime lo siguiente:

```
=== TAREA: roomtype :: SimpleNaiveBayesClassifier ===
Confusion Matrix:
ConfusionMatrix{linearizedMatrix={shared_room={shared_room=2, private_room=2}, entire_home_apt={2=20, shared_room=16, entire_home_apt=316, private_room=24}, private_room={2=10, shared_room=4, entire_home_apt=78, private_room=62}}, avgClassificationTime=29.89325842696629, numberOfEvaluatedDocs=534}
Clase: shared_room
  Precision(shared_room) = 0.09090909090909091
  Recall(shared_room)    = 0.5
  F1(shared_room)        = 0.15384615384615385
Clase: entire_home_apt
  Precision(entire_home_apt) = 0.8020304568527918
  Recall(entire_home_apt)    = 0.8404255319148937
  F1(entire_home_apt)        = 0.8207792207792207
Clase: private_room
  Precision(private_room) = 0.7045454545454546
  Recall(private_room)    = 0.4025974025974026
  F1(private_room)        = 0.5123966942148761
```

Obtenemos un rendimiento moderado: funciona bien para entire\_home y private\_room con una precisión de 0.8 y 0.74 respectivamente, es decir, las suele distinguir bien, pero el recall de private\_room es más bajo, es decir, el modelo aún tiene dificultades distinguiendo las clases.

En el caso del clasificador BM25 observamos:

```

=== TAREA: roomtype :: BM25NBClassifier ===
Confusion Matrix:
ConfusionMatrix{linearizedMatrix={shared_room={2=2, shared_room=2}, entire_home_apt={2=376}, private_room={2=154}}, avgClassification
38577, numberOfEvaluatedDocs=534}
Clase: shared_room
  Precision(shared_room) = 1.0
  Recall(shared_room)    = 0.5
  F1(shared_room)        = 0.6666666666666666
Clase: entire_home_apt
  Precision(entire_home_apt) = 0.0
  Recall(entire_home_apt)    = 0.0
  F1(entire_home_apt)        = 0.0
Clase: private_room
  Precision(private_room) = 0.0
  Recall(private_room)    = 0.0
  F1(private_room)        = 0.0

```

Vemos que sigue sin funcionar bien para esta tarea, colapsando todas las predicciones en una sola etiqueta.

Ocurre algo parecido con KNN:

```

=== TAREA: roomtype :: KNearestNeighborClassifier ===
Confusion Matrix:
ConfusionMatrix{linearizedMatrix={shared_room={private_room=4}, entire_home_apt={private_room=376}, private_room={private_room=154}}, avgClassification
Time=0.8820224719101124, numberOfEvaluatedDocs=534}
Clase: shared_room
  Precision(shared_room) = 0.0
  Recall(shared_room)    = 0.0
  F1(shared_room)        = 0.0
Clase: entire_home_apt
  Precision(entire_home_apt) = 0.0
  Recall(entire_home_apt)    = 0.0
  F1(entire_home_apt)        = 0.0
Clase: private_room
  Precision(private_room) = 0.2883895131086142
  Recall(private_room)    = 1.0
  F1(private_room)        = 0.4476744186046511
Docs evaluados = 534

```

Tiende a predecir la clase mayoritaria, obteniendo un 100% de recall para esta, pero a costo de obtener un 0% en las demás. Esto se puede entender por el desequilibrio de los datos, sin embargo, intentamos un último ajuste de los datos.

Para valores  $k=7$  y  $k=10$  seguimos obteniendo valores muy similares y al intentar ajustar el porcentaje de documentos de entrenamiento al 80% las métricas empeoran. Por todo esto, concluimos que para esta tarea el clasificador más apropiado es SimpleNaiveBayes que, a pesar del desequilibrio de los datos, consigue estimar de manera bastante apropiada las clases más mayoritarias.

## 2. Análisis para neighbourhood

Al comenzar a probar clasificadores para el análisis de esta tarea observamos que pasándole al datasetSplitter un EnglishAnalyzer tokeniza las clases, dando lugar a etiquetas como angel (City Of Los Angeles) o area (Incorporated Areas), por lo que decidimos usar un PerFieldAnalyzer que use EnglishAnalyzer para el texto y KeyWordAnalyzer para la clase.

Analizamos un total de 273 documentos de los cuales: 181 son Los Ángeles, 75 other cities y 17 son unincorporated areas. Es decir, el conjunto de datos es bastante desequilibrado.

Así, usando 70% de documentos para el entrenamiento y definiendo  $k=5$  :

- Clasificador SimpleNaiveBayes:

```
=== TAREA: neighbourhood :: SimpleNaiveBayesClassifier ===
Confusion Matrix:
ConfusionMatrix{linearizedMatrix={unincorporated_areas={unincorporated_areas=8, city_of_los_angeles=1, 34.091907218739856=8}, city_of_los_angeles={unincorporated_areas=38, city_of_los_angeles=48, other_cities=10, 34.091907218739856=85}, other_cities={unincorporated_areas=25, city_of_los_angeles=6, other_cities=12, 34.091907218739856=32}}, avgClassificationTime=6.688644688644689, numberOfEvaluatedDocs=273}
Clase: unincorporated_areas
Precision(unincorporated_areas) = 0.11267605633802817
Recall(unincorporated_areas) = 0.47058823529411764
F1(unincorporated_areas) = 0.18181818181818182
Clase: city_of_los_angeles
Precision(city_of_los_angeles) = 0.8727272727272727
Recall(city_of_los_angeles) = 0.26519337016574585
F1(city_of_los_angeles) = 0.40677966101694907
Clase: other_cities
Precision(other_cities) = 0.5454545454545454
Recall(other_cities) = 0.16
F1(other_cities) = 0.24742268041237112
```

El modelo no colapsa todo en una única clase y es bastante preciso a la hora de predecir city\_of\_los\_angeles, sin embargo se queda bastante corto en recall.

- Clasificador BM25:

```
=== TAREA: neighbourhood :: BM25NBClassifier ===
Confusion Matrix:
ConfusionMatrix{linearizedMatrix={unincorporated_areas={34.091907218739856=17}, city_of_los_angeles={34.091907218739856=181}, other_cities={34.091907218739856=75}}, avgClassificationTime=5.227106227106227, numberOfEvaluatedDocs=273}
Clase: unincorporated_areas
Precision(unincorporated_areas) = 0.0
Recall(unincorporated_areas) = 0.0
F1(unincorporated_areas) = 0.0
Clase: city_of_los_angeles
Precision(city_of_los_angeles) = 0.0
Recall(city_of_los_angeles) = 0.0
F1(city_of_los_angeles) = 0.0
Clase: other_cities
Precision(other_cities) = 0.0
Recall(other_cities) = 0.0
F1(other_cities) = 0.0
```

Es inutilizable para esta tarea con estos ajustes.

- Clasificador KNN:

```
=== TAREA: neighbourhood :: KNearestNeighborClassifier ===
Confusion Matrix:
ConfusionMatrix{linearizedMatrix={unincorporated_areas={city_of_los_angeles=17}, city_of_los_angeles={city_of_los_angeles=181}, other_cities={city_of_los_angeles=75}}, avgClassificationTime=0.6446886446886447, numberOfEvaluatedDocs=273}
Clase: unincorporated_areas
Precision(unincorporated_areas) = 0.0
Recall(unincorporated_areas) = 0.0
F1(unincorporated_areas) = 0.0
Clase: city_of_los_angeles
Precision(city_of_los_angeles) = 0.663003663003663
Recall(city_of_los_angeles) = 1.0
F1(city_of_los_angeles) = 0.7973568281938327
Clase: other_cities
Precision(other_cities) = 0.0
Recall(other_cities) = 0.0
F1(other_cities) = 0.0
Docs evaluados = 273
```

Sólo consigue predecir la clase mayoritaria.

A continuación, probamos a ajustar  $k = 8$  y en lugar de description usamos neighbourhood\_overview como campo de texto, además de cambiar a StandardAnalyzer. Los resultados apenas varían y no nos permiten sacar conclusiones nuevas.

En última instancia, probamos el clasificador KFuzzy con  $k = 8$  y obtenemos :

```
=== TAREA: neighbourhood :: KFuzzyClassifier ===
Confusion Matrix:
ConfusionMatrix{linearizedMatrix={unincorporated_areas={unincorporated_areas=3, city_of_los_angeles=9}, city_of_los_angeles={city_of_los_angeles=146, other_cities=11}, other_cities={city_of_los_angeles=38, other_cities=20}}, avgClassificationTime=7.5198237885462555, numberOfEvaluatedDocs=227}
Clase: unincorporated_areas
Precision(unincorporated_areas) = 1.0
Recall(unincorporated_areas) = 0.25
F1(unincorporated_areas) = 0.4
Clase: city_of_los_angeles
Precision(city_of_los_angeles) = 0.7564766839378239
Recall(city_of_los_angeles) = 0.9299363057324841
F1(city_of_los_angeles) = 0.8342857142857144
Clase: other_cities
Precision(other_cities) = 0.6451612903225806
Recall(other_cities) = 0.3448275862068966
F1(other_cities) = 0.44943820224719105
Docs evaluados = 227
```

Este nos da nuestros mejores resultados hasta ahora :

- De todos los documentos que clasifica como City Of Los Ángeles, acierta en un 76% y consigue clasificar el 93% de los que realmente lo son.
- Con Other Cities acierta cerca de 65% aunque tiene un recall más bajo (34%)
- Por último, con Unincorporated Areas siempre acierta al predecirlas, aunque solo consigue clasificar 3 de las 12 totales que hay.

Así, podemos concluir que este clasificador, utilizando como campo de texto neighbourhood\_overview y StandardAnalyzer, es la mejor apuesta para clasificar por el campo neighbourhood\_group a pesar de las limitaciones que presenta debido al sesgo de los datos.

### 3. Análisis para property\_type

Para este análisis hemos directamente descartado el uso del clasificador KNN y hemos optado mejor por usar Kfuzzy. Así, usando el campo de texto description, k=8 y el StandardAnalyzer (dentro de un PerFieldAnalyzer) tenemos:

Resultados para el clasificador SimpleNaiveBayer:

```
=== TAREA: proptype :: SimpleNaiveBayesClassifier ===
Confusion Matrix:
ConfusionMatrix({linearizedMatrix={rental unit={other=7, townhouse=2, bungalow=11, guest=17, condo=7, villa=4, guesthouse=9, rental=16, home=1}, other={
other=2, townhouse=1, bungalow=2, guest=2, guesthouse=1, villa=1}, townhouse={other=1, townhouse=2, bungalow=2, villa=1}, loft={loft=2, townhouse=1, bu
ngalow=1, guesthouse=1, home=1}, guest suite={other=1, guest=6, guesthouse=4}, bungalow={bungalow=3, guest=5, guesthouse=3, home=1}, hotel={hotel=4, gu
esthouse=1}, condo={other=1, townhouse=3, loft=2, hotel=1, guest=3, condo=1, villa=2, guesthouse=2}, guesthouse={other=2, bungalow=2, hotel=1, guest=6,
guesthouse=10, villa=3, home=2}, villa={guest=2, villa=3}, home={other=11, townhouse=3, loft=1, bungalow=13, hotel=1, guest=28, guesthouse=24, villa=1
5, home=7}}, avgClassificationTime=10.503676470588236, numberOfEvaluatedDocs=272}
Class: rental unit
  Precision(rental unit) = 0.0
  Recall(rental unit)   = 0.0
  F1(rental unit)       = 0.0
Class: other
  Precision(other) = 0.08
  Recall(other)   = 0.2222222222222222
  F1(other)       = 0.11764705882352941
Class: townhouse
  Precision(townhouse) = 0.16666666666666666
  Recall(townhouse)   = 0.3333333333333333
  F1(townhouse)       = 0.2222222222222222
Class: loft
  Precision(loft) = 0.4
  Recall(loft)   = 0.3333333333333333
  F1(loft)       = 0.3636363636363636
Class: guest suite
  Precision(guest suite) = 0.0
  Recall(guest suite)   = 0.0
  F1(guest suite)       = 0.0
Class: bungalow
  Precision(bungalow) = 0.08823529411764706
  Recall(bungalow)   = 0.25
  F1(bungalow)       = 0.13043478260869565
Class: hotel
  Precision(hotel) = 0.5714285714285714
  Recall(hotel)   = 0.8
  F1(hotel)       = 0.6666666666666666
Class: condo
  Precision(condo) = 0.125
  Recall(condo)   = 0.06666666666666667
  F1(condo)       = 0.08695652173913045
```

```

Clase: guesthouse
Precision(guesthouse) = 0.181818181818182
Recall(guesthouse)    = 0.38461538461538464
F1(guesthouse)        = 0.2469135802469136
Clase: villa
Precision(villa)      = 0.10344827586206896
Recall(villa)         = 0.6
F1(villa)             = 0.17647058823529413
Clase: home
Precision(home)       = 0.5833333333333334
Recall(home)          = 0.06796116504854369
F1(home)              = 0.12173913043478261
Clase: guest
Precision(guest)      = 0.0
Recall(guest)         = 0.0
F1(guest)             = 0.0
Clase: rental
Precision(rental)     = 0.0
Recall(rental)        = 0.0
F1(rental)           = 0.0
Docs evaluados = 272
-----

```

Observamos que el modelo predice bastante bien hotel, probablemente por ser una palabra más rara en las descripciones. Sin embargo, la mayoría de categorías presentan un F1 bajo, es decir, el modelo las confunde mucho.

En el caso del clasificador BM25:

```

Clase: rental unit
Precision(rental unit) = 0.0
Recall(rental unit)   = 0.0
F1(rental unit)       = 0.0
Clase: other
Precision(other)      = 0.0
Recall(other)         = 0.0
F1(other)             = 0.0
Clase: townhouse
Precision(townhouse)  = 0.034482758620689655
Recall(townhouse)     = 0.16666666666666666
F1(townhouse)        = 0.05714285714285715
Clase: loft
Precision(loft)       = 0.06896551724137931
Recall(loft)          = 0.3333333333333333
F1(loft)              = 0.1142857142857143
Clase: guest suite
Precision(guest suite) = 0.0
Recall(guest suite)    = 0.0
F1(guest suite)        = 0.0
Clase: bungalow
Precision(bungalow)   = 0.0
Recall(bungalow)      = 0.0
F1(bungalow)          = 0.0
Clase: hotel
Precision(hotel)       = 1.0
Recall(hotel)          = 1.0
F1(hotel)              = 1.0
Clase: condo
Precision(condo)       = 0.0
Recall(condo)          = 0.0
F1(condo)              = 0.0
Clase: guesthouse
Precision(guesthouse) = 0.0
Recall(guesthouse)    = 0.0
F1(guesthouse)        = 0.0

```

```

Clase: villa
Precision(villa) = 0.025380710659898477
Recall(villa)    = 1.0
F1(villa)        = 0.04950495049504951
Clase: home
Precision(home)  = 0.0
Recall(home)     = 0.0
F1(home)         = 0.0
Docs evaluados = 272

```

Al igual que en caso anterior se predice de forma muy eficaz la etiqueta de hotel, y también nos encontramos con que villa tiene un recall del 100%, pero la precisión apenas es de un 2%, es decir, tiende a clasificar las etiquetas que no conoce directamente como villa. La mayoría de clases presentan un  $F1=0$ , por lo que podemos concluir que no es un buen clasificador para esta tarea.

Por último para KFuzzy observamos:

```
Clase: rental unit
Precision(rental unit) = 0.5
Recall(rental unit)    = 0.5675675675675675
F1(rental unit)        = 0.5316455696202531
Clase: other
Precision(other)       = 0.3333333333333333
Recall(other)          = 0.1111111111111111
F1(other)              = 0.1666666666666666
Clase: townhouse
Precision(townhouse)   = 0.0
Recall(townhouse)      = 0.0
F1(townhouse)          = 0.0
Clase: loft
Precision(loft)        = 1.0
Recall(loft)           = 0.1666666666666666
F1(loft)               = 0.2857142857142857
Clase: guest suite
Precision(guest suite) = 0.0
Recall(guest suite)    = 0.0
F1(guest suite)        = 0.0
Clase: bungalow
Precision(bungalow)    = 0.0
Recall(bungalow)       = 0.0
F1(bungalow)           = 0.0
Clase: hotel
Precision(hotel)       = 1.0
Recall(hotel)          = 0.8
F1(hotel)              = 0.8888888888888889
Clase: condo
Precision(condo)       = 0.0
Recall(condo)          = 0.0
F1(condo)              = 0.0
Clase: guesthouse
Precision(guesthouse)  = 0.25
Recall(guesthouse)     = 0.23076923076923078
F1(guesthouse)         = 0.24000000000000002
Clase: villa
Precision(villa)       = 0.0
Recall(villa)          = 0.0
F1(villa)              = 0.0
Clase: home
Precision(home)        = 0.5034965034965035
Recall(home)           = 0.6990291262135923
F1(home)               = 0.5853658536585367
Docs evaluados = 272
```

Las etiquetas con F1=0 son aquellas que presentan muy pocos ejemplos entre los documentos, lo cuál es comprensible, pero el resto de etiquetas se predicen con bastantes buenos resultados, de hecho, fijándonos en la matriz de confusión:

```
=== TAREA: proptype :: KFuzzyClassifier ===
Confusion Matrix:
ConfusionMatrix{linearizedMatrix={rental unit={rental unit=42, guest suite=1, bungalow=1, condo=1, guesthouse=1, home=28}, other={other=1, bungalow=2, guesthouse=1, home=5}, townhouse={rental unit=4, other=1, home=1}, loft={rental unit=3, loft=1, guesthouse=1, home=1}, guest suite={guesthouse=1, home=10}, bungalow={rental unit=4, guesthouse=2, home=6}, hotel={hotel=4, guesthouse=1}, condo={rental unit=8, townhouse=1, bungalow=1, guesthouse=1, home=4}, guesthouse={rental unit=6, guest suite=2, bungalow=1, guesthouse=6, home=11}, villa={home=5}, home={rental unit=17, other=1, guest suite=2, bungalow=1, guesthouse=10, home=72}}, avgClassificationTime=5.400735294117647, numberOfEvaluatedDocs=272}
```

Al sumar los aciertos vemos que son 126 del total de 272 documentos, lo cuál se acerca al 50% siendo los mejores resultados obtenidos hasta el momento.

Si reducimos ligeramente k al valor 5 observamos una pequeña pérdida para home pero también una ligera mejora para el resto de etiquetas más mayoritarias, logrando además F1=1 para hotel.

Por todo esto, concluimos que la mejor apuesta de clasificador para esta tarea es el KFuzzy con k=5 y un StandardAnalyzer (probamos también EnglishAnalyzer pero bajan todas las métricas).

## 4. Análisis para review\_scores\_ratings

Continuamos con review\_scores\_ratings. tenemos la misma configuración como antes. El resultado de SimpleNaiveBayer:

```
=== TAREA: rating :: SimpleNaiveBayesClassifier ===
Confusion Matrix:
ConfusionMatrix{linearizedMatrix={high={high=24, excellent=10, medium=22}, excellent={high=22, excellent=62, medium=108}, medium={high=1, medium=3}},
Clase: high
  Precision(high) = 0.5106382978723404
  Recall(high)    = 0.42857142857142855
  F1(high)        = 0.46601941747572817
Clase: excellent
  Precision(excellent) = 0.8611111111111112
  Recall(excellent)    = 0.32291666666666667
  F1(excellent)        = 0.46969696969696967
Clase: medium
  Precision(medium) = 0.022556390977443608
  Recall(medium)    = 0.75
  F1(medium)        = 0.043795620437956206
Docs evaluados = 252
```

Este classifier funciona, pero no funciona bien porque la F1 score de cada clase es muy corto. Para los demás de clasificadores, el mejor rendimiento es de KFUZZYClassifier, pero falla en la predicción de la clase medium. Además, hay que tener en cuenta que los datos son bastante desequilibrado, hay 74~76% de clase excellent, así que asumimos el rendimiento en predicción de las otras clases sean más importantes en evaluar el rendimiento real.

```
=== TAREA: rating :: KFUZZYClassifier ===
Confusion Matrix:
ConfusionMatrix{linearizedMatrix={high={high=13, excellent=43}, excellent={high=9, excellent=183}, medium={high=2, excellent=2}},
Clase: high
  Precision(high) = 0.5416666666666666
  Recall(high)    = 0.23214285714285715
  F1(high)        = 0.325
Clase: excellent
  Precision(excellent) = 0.8026315789473685
  Recall(excellent)    = 0.953125
  F1(excellent)        = 0.8714285714285714
Clase: medium
  Precision(medium) = 0.0
  Recall(medium)    = 0.0
  F1(medium)        = 0.0
Docs evaluados = 252
```

Nos parece que el uso de description como el campo de texto no sirve bien para la clasificación de este campo. así que probamos lo que puede ser relevantes con review\_scores\_ratings, como “host\_about”, “number\_of\_reviews”, y “price”.

host\_about:

```
=== TAREA: rating :: SimpleNaiveBayesClassifier ===
Confusion Matrix:
ConfusionMatrix{linearizedMatrix={high={high=14, excellent=17, medium=30}, excellent={high=15, excellent=54, medium=143}, medium={medium=6}},
Clase: high
  Precision(high) = 0.4827586206896552
  Recall(high)    = 0.22950819672131148
  F1(high)        = 0.3111111111111111
Clase: excellent
  Precision(excellent) = 0.7605633802816901
  Recall(excellent)    = 0.25471698113207547
  F1(excellent)        = 0.3816254416961131
Clase: medium
  Precision(medium) = 0.0335195530726257
  Recall(medium)    = 1.0
  F1(medium)        = 0.06486486486486485
Docs evaluados = 279
```

```

=== TAREA: rating :: BM25NBClassifier ===
Confusion Matrix:
ConfusionMatrix{linearizedMatrix={high={high=3, medium=58}, excellent={high=1, excellent=1, medium=210}, medium={medium=6}},
Clase: high
  Precision(high) = 0.75
  Recall(high)    = 0.04918032786885246
  F1(high)        = 0.0923076923076923
Clase: excellent
  Precision(excellent) = 1.0
  Recall(excellent)    = 0.0047169811320754715
  F1(excellent)        = 0.009389671361502346
Clase: medium
  Precision(medium) = 0.021897810218978103
  Recall(medium)    = 1.0
  F1(medium)        = 0.04285714285714286
Docs evaluados = 279

```

```

=== TAREA: rating :: KFuzzyClassifier ===
Confusion Matrix:
ConfusionMatrix{linearizedMatrix={high={high=15, excellent=45}, excellent={high=15, excellent=194}, medium={excellent=6}},
Clase: high
  Precision(high) = 0.5
  Recall(high)    = 0.25
  F1(high)        = 0.3333333333333333
Clase: excellent
  Precision(excellent) = 0.7918367346938775
  Recall(excellent)    = 0.9282296650717703
  F1(excellent)        = 0.8546255506607928
Clase: medium
  Precision(medium) = 0.0
  Recall(medium)    = 0.0
  F1(medium)        = 0.0
Docs evaluados = 275

```

KFuzzyClassifier funciona más o menos igual, sigue fallando en classificar la clase medium. El rendimiento de SimpleNaiveBayesClassifier ha empeorado, el rendimiento de BM25NBClassifier ha mejorado un poco, pero no mucho. Luego, vamos a comparar el rendimiento de SimpleNaiveBayesClassifier, salvo que los otros clasificadores tengan un gran cambio de rendimiento porque es el único que posiblemente pueda tener un mejor resultado en predecir la clase medium correctamente (según el resultado arriba).

number\_of\_reviews

```

=== TAREA: rating :: SimpleNaiveBayesClassifier ===
Confusion Matrix:
ConfusionMatrix{linearizedMatrix={high={high=1, excellent=60}, excellent={high=5, excellent=207}, medium={excellent=6}},
Clase: high
  Precision(high) = 0.16666666666666666
  Recall(high)    = 0.01639344262295082
  F1(high)        = 0.02985074626865672
Clase: excellent
  Precision(excellent) = 0.7582417582417582
  Recall(excellent)    = 0.9764150943396226
  F1(excellent)        = 0.8536082474226804
Clase: medium
  Precision(medium) = 0.0
  Recall(medium)    = 0.0
  F1(medium)        = 0.0
Docs evaluados = 279

```

price

```

=== TAREA: rating :: SimpleNaiveBayesClassifier ===
Confusion Matrix:
ConfusionMatrix{linearizedMatrix={high={high=6, excellent=46}, excellent={high=2, excellent=179}, medium={excellent=2}},
Clase: high
  Precision(high) = 0.75
  Recall(high)    = 0.11538461538461539
  F1(high)        = 0.19999999999999998
Clase: excellent
  Precision(excellent) = 0.788546255506608
  Recall(excellent)    = 0.988950276243094
  F1(excellent)        = 0.877450980392157
Clase: medium
  Precision(medium) = 0.0
  Recall(medium)    = 0.0
  F1(medium)        = 0.0
Docs evaluados = 235

```

Los resultados de SimpleNaiveBayesClassifier han mejorado, usando number\_of\_reviews o price, pero el mejor rendimiento sigue siendo lo del KFuzzyClassifier, con mejores F1 scores en predecir las clases high y excellent.

## 5. Trabajo en grupo

Elena Abreu : elaboración del código, análisis de las tareas: room\_type, neighbourhood y property\_type

Tsan-Yu Wu : análisis de la tarea review\_scores\_rating